

# **EggMarket: A Modern Digital Marketplace and Supply Chain Management System**

**Prepared By:**

*CHRISTIAN P. CATURAN*

**Course/Subject:**

*IT312L – System Integration and Architecture 2*

**Instructor:**

*Jeremiah C. Dela Cruz*

**Date Submitted:**

*22 May 2026*

## **I. Introduction**

### **Background of the Project**

The agricultural sector, specifically the poultry and egg industry, remains a vital part of the global food supply chain. However, the traditional trade route often involves multiple layers of middlemen, which increases the retail price for consumers while reducing the profit margins for local farmers.

### **Purpose of the System**

The Egg Market Management System (EggMarket) is a web-based platform designed to bridge the gap between farmers (suppliers) and customers. It centralizes trade operations into a single digital terminal, allowing for direct transactions, real-time inventory tracking, and transparent pricing.

### **Importance of the Project**

By digitizing the marketplace, the project empowers small-to-medium scale farmers to reach a wider audience without relying on third-party distributors. It provides a structured environment where quality, freshness, and traceability are prioritized.

### **Problems the System Aims to Solve**

1. **Inefficient Supply Chain:** Reducing the delay between harvest and delivery.
2. **Lack of Transparency:** Providing customers with clear information on farm origins and harvest dates.
3. **Manual Inventory Management:** Automating stock tracking for farmers to prevent overselling.
4. **Geographical Barriers:** Using GPS technology to optimize logistics for delivery nodes.

## **II. Objectives of the System**

### **General Objective**

To develop a modern, responsive, and secure digital marketplace that facilitates direct-to-consumer egg trade while providing comprehensive management tools for farmers and administrators.

### **Specific Objectives**

1. To implement a robust Role-Based Access Control (RBAC) system for Admins, Farmers, and Customers.
2. To create a dynamic marketplace with advanced search and category filtering.
3. To integrate real-time geolocation services for precise delivery and farm locating.
4. To provide analytical dashboards with interactive charts for sales and inventory monitoring.
5. To automate notifications for low-stock levels and new order arrivals.

## **III. Scope and Limitation of the System**

The system provides secure authentication through a registration and login module with role-based redirection to ensure users are directed to their appropriate interfaces. It includes a marketplace that allows customers to browse eggs categorized as Chicken, Duck, and Organic, with pricing available per unit such as per tray or per dozen. A Farmer Suite is also implemented, offering full product management capabilities including create, read, update, and delete (CRUD) operations, image uploading for product listings, and fulfillment tracking with statuses such as Pending, Processing, On the Way, and Delivered. For administrators, an Admin Suite is provided to handle user moderation, category management, and platform-wide financial oversight to maintain system integrity and control. The system also incorporates real-time features through AJAX-based system heartbeat functionality, enabling instant notifications and live updates. Additionally, logistics support is integrated through mapping features that allow visualization and identification of both customer and supplier locations for improved delivery coordination.

The system currently supports only Cash on Delivery (COD) as its available payment method, with no integration for digital or online payment gateways. It also requires a stable internet connection to properly function, particularly for real-time API synchronization and map rendering features. Furthermore, the platform is strictly web-based in its current phase, meaning that no dedicated mobile application version is available at this time.

#### **IV. System Description**

##### **Overview of the System**

EggMarket is an enterprise-grade digital exchange built on the Model-View-Controller (MVC) architecture. It uses a professional SaaS-style design to provide a high-fidelity user experience.

##### **Major Modules / Features**

1. **Trade Terminal:** The primary marketplace where transactions occur.
2. **Inventory Control:** Automated stock management with critical level alerts.
3. **Order Fulfillment:** A workflow engine for managing the lifecycle of a purchase.
4. **Weather Intel:** Live environmental monitoring using public APIs to warn farmers of potential heat stress for livestock.

##### **User Roles and Functionalities**

- **Admin:** Manages user lifecycles, approves new farmers, audits all platform transactions, and configures global trade settings.
- **Farmer:** Lists products, uploads images, tracks sales analytics, fulfills orders, and receives system alerts.
- **Customer:** Browses products, manages a persistent shopping cart, tracks order status, and provides product reviews.

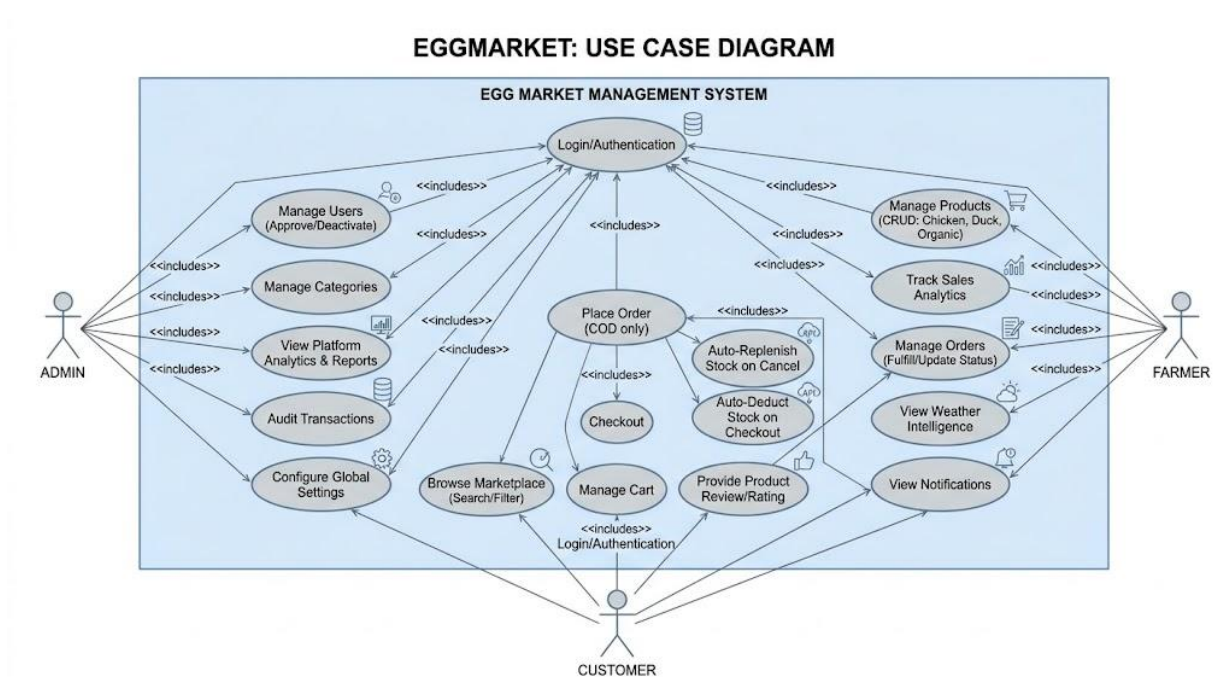
## V. Technologies Used

- **Programming Language:** PHP 8.2 (Backend), JavaScript (Frontend Logic).
- **Frameworks:** Laravel 12 (Core Engine), Tailwind CSS (Styling).
- **Database:** MySQL 8.0 (Relational Data Storage).
- **Libraries:**
  - **ApexCharts:** For professional data visualization.
  - **Leaflet.js:** For GPS mapping and geolocation services.
  - **Lucide React / FontAwesome 6:** For iconography.
- **Deployment:** Docker (Containerization for App, DB, and Nginx).

## VI. System Design

### Use Case Diagram

The EggMarket Management System Use Case Diagram outlines the interactions between three primary actors: the Admin, Farmer, and Customer. A central Login/Authentication requirement secures nearly all operations within the system boundary, ensuring that users are verified before performing key actions. The Admin oversees backend governance by managing users, categories, global settings, and transaction audits. The Farmer handles inventory vendor actions—performing CRUD operations on chicken, duck, and organic egg listings—while tracking sales analytics, fulfilling orders, and monitoring weather intelligence. Meanwhile, the Customer drives the marketplace by browsing products, managing their cart, and executing a Cash on Delivery (COD) checkout workflow. This ordering process seamlessly connects to automated inventory features that automatically deduct stock upon checkout and replenish it if an order is canceled, while also allowing customers to view notifications and leave product reviews.

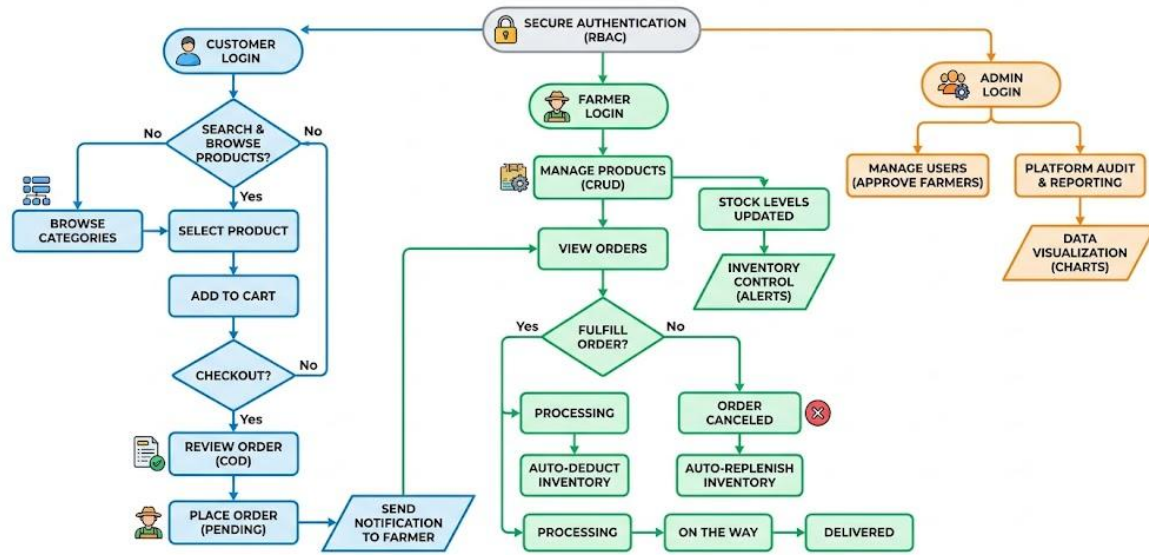


**Figure 1: Use Case Diagram**

### Flowchart Diagram

This flowchart maps the operational lifecycle and data flow of the EggMarket platform, tracking how Role-Based Access Control (RBAC) securely splits the system into three distinct user pathways. On the blue Customer path, users search products or browse categories to select items, add them to a cart, review their Cash on Delivery (COD) details, and place a pending order, which instantly triggers a notification to the vendor. This connects directly to the green Farmer workflow, where sellers view incoming orders to either process and fulfill them—which auto-deducts inventory and moves the order through processing, transit, and delivery statuses—or cancel them, which auto-replenishes the stock and updates inventory control alerts. Concurrently, the orange Admin pathway oversees backend platform integrity by managing user accounts, approving farmers, and running platform audits that feed into visual data reporting charts.

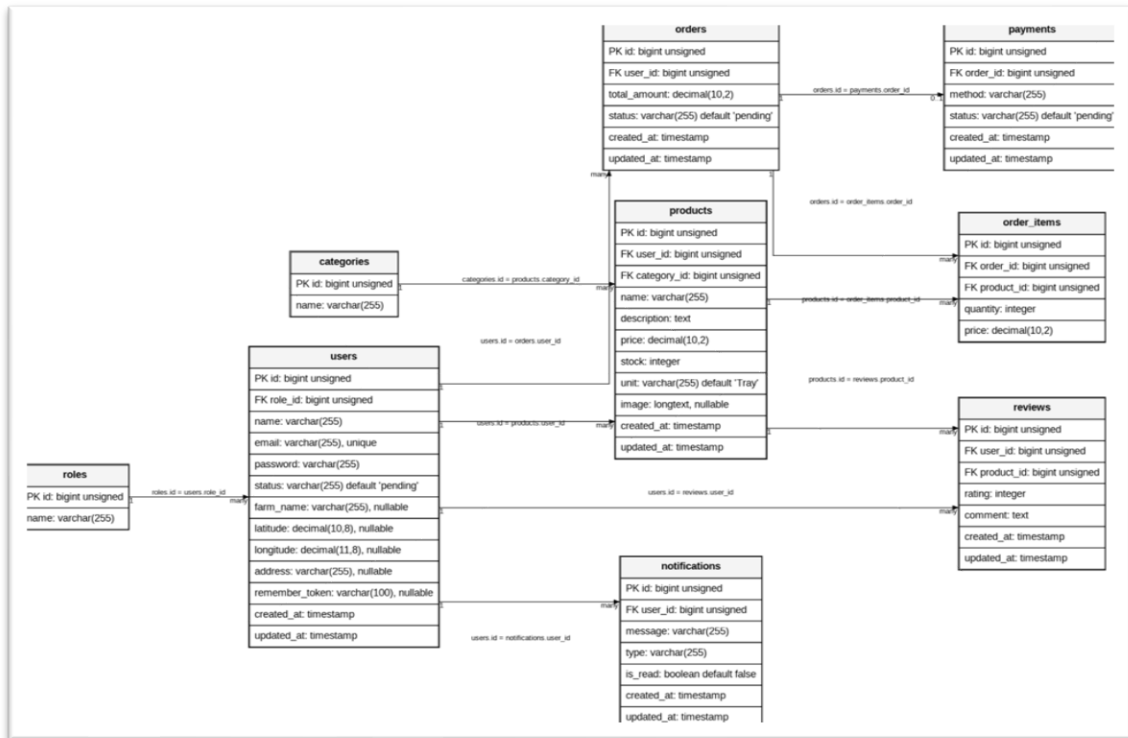
## EGGMARKET: DIGITAL MARKETPLACE & SUPPLY CHAIN FLOW



**Figure 2:** Flowchart Diagram

### ER Diagram

The EggMarket Entity-Relationship (ER) Diagram defines the relational database structure for the platform using Crow's Foot notation, organized around core user roles and transactional data. At its foundation, the users table establishes account identities—such as Admin, Farmer, or Customer—by linking a mandatory one-to-many relationship with the roles table. The supply side is managed via the products table, which holds inventory data like pricing, stock levels, and unit types (Tray, Dozen), mapped to specific categories (Chicken, Duck, Organic) and owned by a farmer user. The transactional core connects users (as customers) to the orders table, which breaks down individual transactions into detailed order\_items linked to the products sold. Finally, the system tracks post-purchase activity and operations through peripheral tables, linking every completed checkout to cash-on-delivery records in payments, broadcasting updates via notifications, and managing customer feedback within the reviews table.



**Figure 3: ER Diagram**

## System Architecture Diagram

The EggMarket System Architecture Diagram maps out the end-to-end deployment infrastructure, illustrating how client requests from the Admin, Farmer, and Customer securely pass through an API Gateway (Nginx Proxy), a Load Balancer, and a Web Application Firewall (WAF). At the core sits a Laravel 12 Application structured around Model-View-Controller (MVC) architecture, which separates responsibilities into a Web/API Controller layer for RBAC authentication, a Business Logic layer for processing marketplace rules, and an Eloquent ORM Data Access layer. This backend application handles internal storage through a Data Tier composed of a MySQL 8.0 relational database, a Redis Cache for session optimization, and Object Storage (like Amazon S3) for product images, while concurrently using Docker containerization for seamless deployment. Furthermore, the core framework communicates externally via API calls to Third-Party Integrations and Services, which pull location data through Leaflet.js, extract public environmental information via a Weather Data API, generate dashboard metrics through ApexCharts, and broadcast SMS or chat alerts using an integrated Notification Engine.

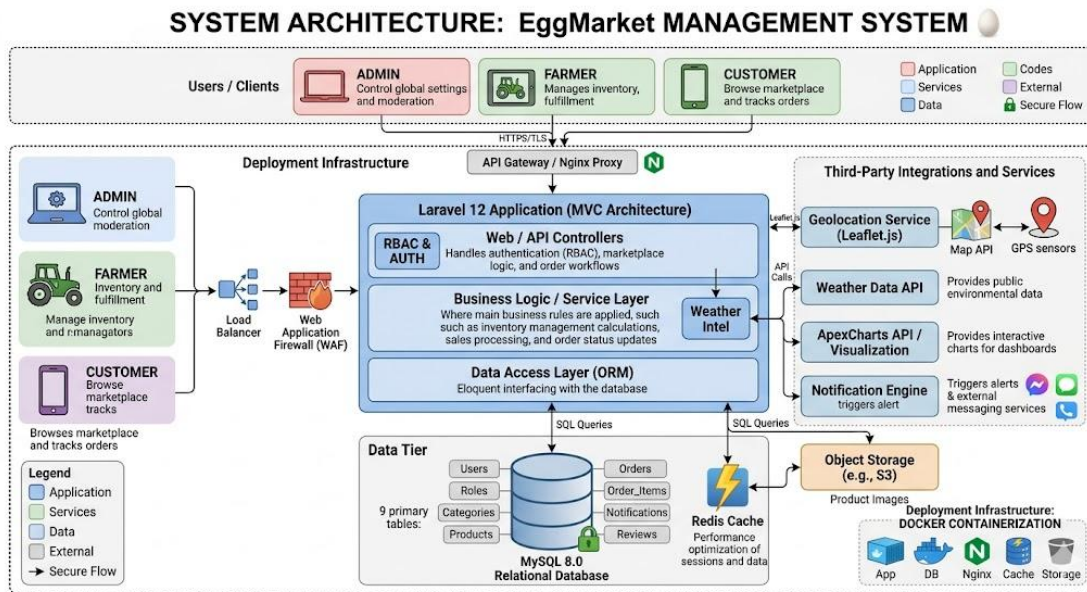


Figure 4: System Architecture Diagram

## Interface Design / Screenshots

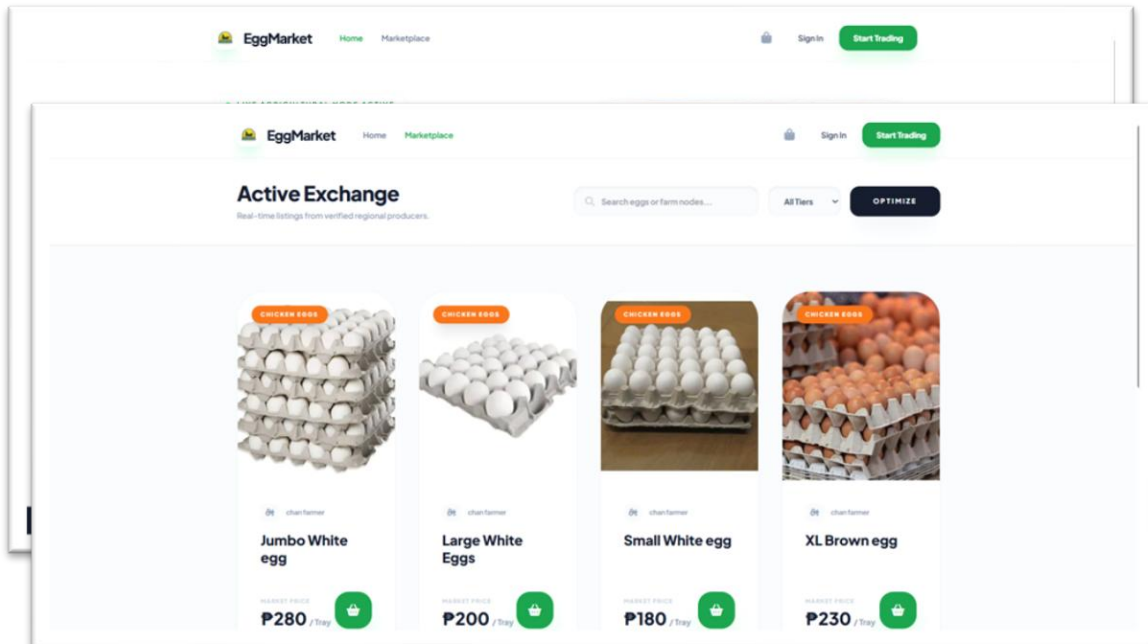
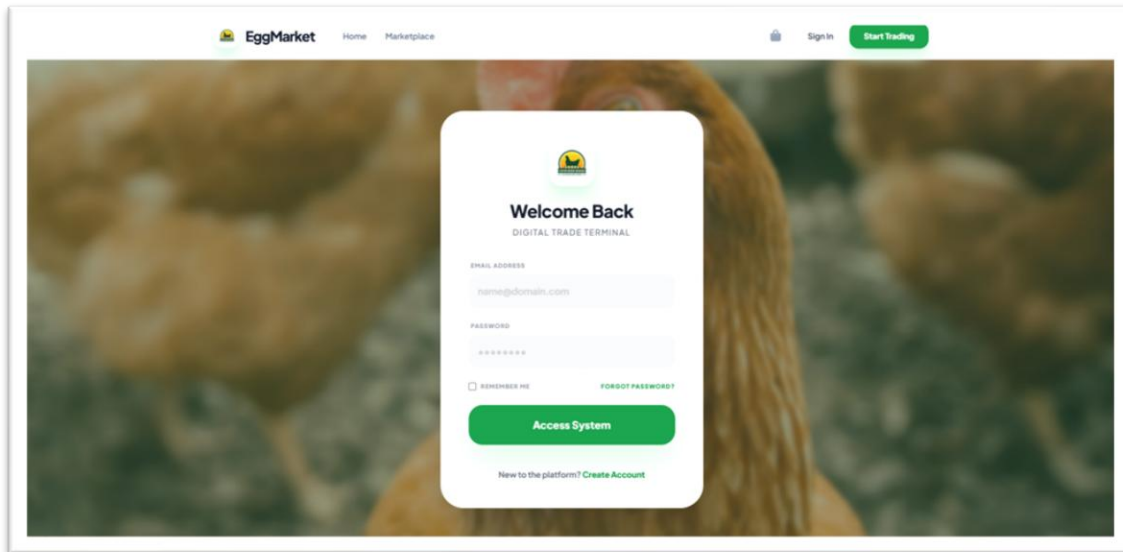


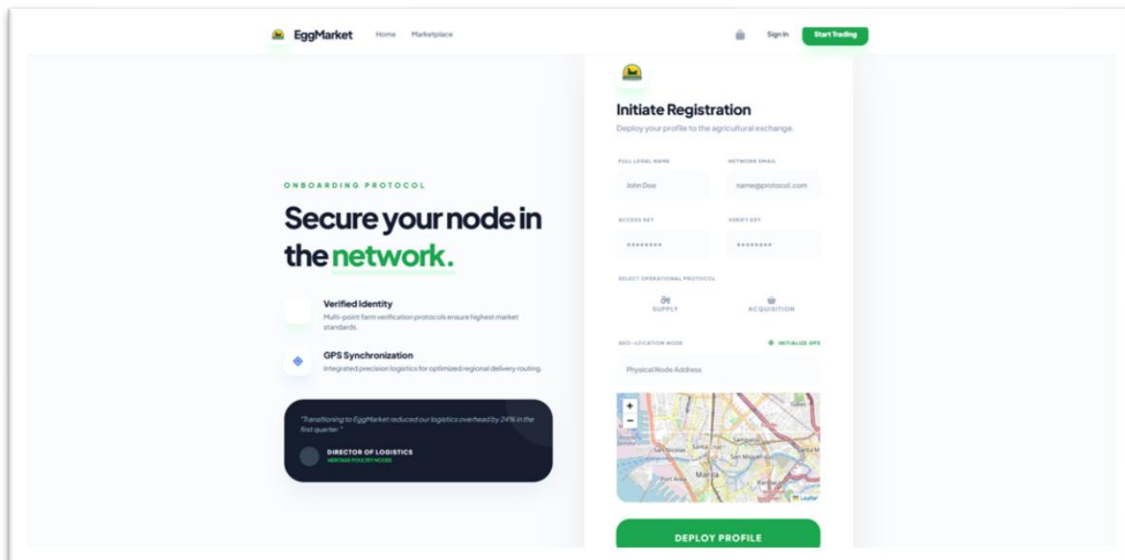
Figure 5: Welcome Page

**Figure 6: Marketplace**



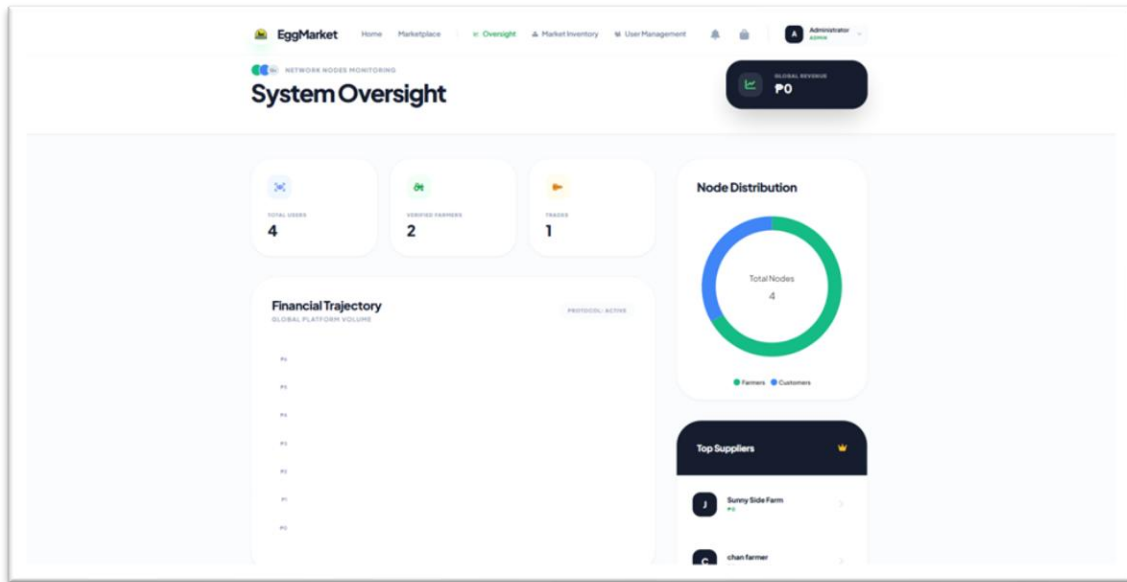
The screenshot shows the EggMarket login interface. At the top, there is a navigation bar with the EggMarket logo, 'Home', and 'Marketplace' links. On the right, there are 'Sign In' and 'Start Trading' buttons. The main content area features a large background image of chickens. In the center, a white login card is displayed with the following elements: the EggMarket logo, the heading 'Welcome Back' and subtitle 'DIGITAL TRADE TERMINAL', an 'EMAIL ADDRESS' field with the placeholder 'name@domain.com', a 'PASSWORD' field with masked characters, a 'Remember Me' checkbox, a 'Forgot Password?' link, a green 'Access System' button, and a link for new users to 'Create Account'.

**Figure 7: Login form**

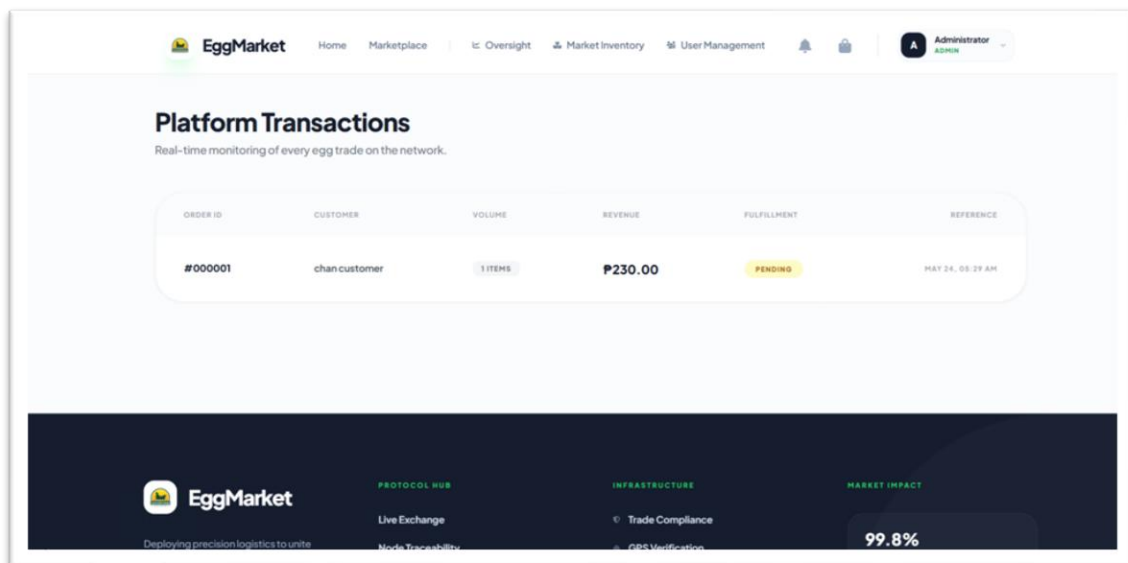


The screenshot displays the EggMarket registration process, split into two panels. The left panel, titled 'ONBOARDING PROTOCOL', has the heading 'Secure your node in the network.' and lists two steps: 'Verified Identity' (Multi-point farm verification protocols ensure highest market standards) and 'GPS Synchronization' (Integrated precision logistics for optimized regional delivery routing). It includes a quote from the 'DIRECTOR OF LOGISTICS' and a dark blue 'INITIALIZE GPS' button. The right panel, titled 'Initiate Registration', prompts the user to 'Deploy your profile to the agricultural exchange.' and contains fields for 'FULL LEGAL NAME' (John Doe), 'NETWORK EMAIL' (name@protocol.com), 'ACCESS KEY' (masked), and 'VERIFY KEY' (masked). Below these are radio buttons for 'SUPPLY' and 'ACQUISITION' protocols, a 'PHYSICAL NODE ADDRESS' field, and a map. A green 'DEPLOY PROFILE' button is at the bottom.

**Figure 8: Register form**



*Figure 9: Admin Dashboard*



*Figure 10: Platform Transactions*

**Inventory Audit**  
Cross-node monitoring of active exchange listings.

Search across all farm nodes...

| REFERENCE | PRODUCE NODE INTELLIGENCE            | ORIGIN FARMER | TIER         | STOCK NODE | UNIT VALUE | PROTOCOL |
|-----------|--------------------------------------|---------------|--------------|------------|------------|----------|
| WD-4      | Jumbo White egg<br>VERIFIED LISTING  | chan farmer   | CHICKEN EGGS | 100        | P280.00    |          |
| WD-2      | Large White Eggs<br>VERIFIED LISTING | chan farmer   | CHICKEN EGGS | 100        | P200.00    |          |

Small

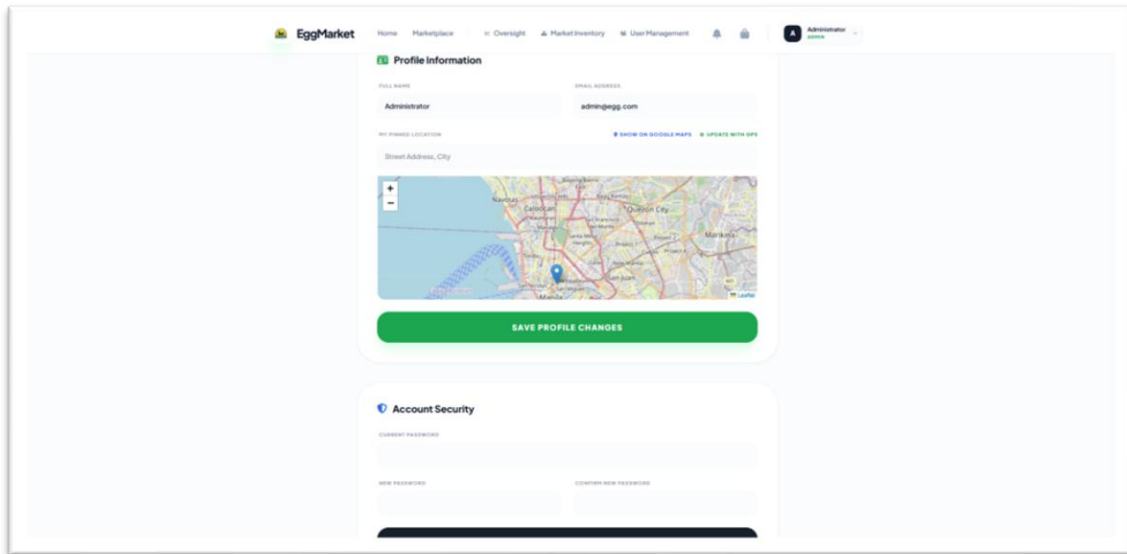
*Figure 11: Admin Inventory*

**User Management**  
Lifecycle governance for all network entities and farm nodes.

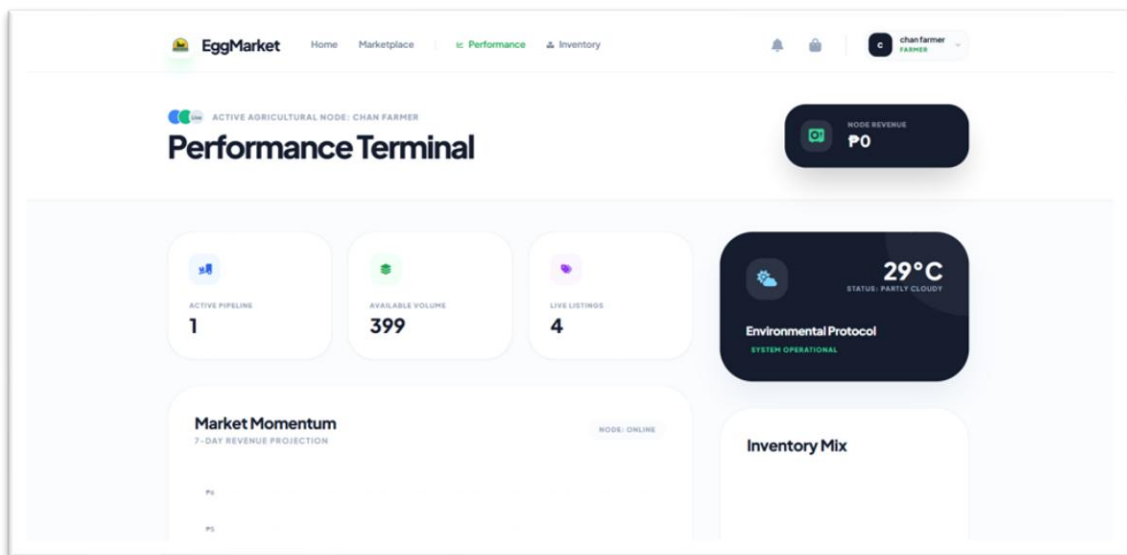
FARMERS CUSTOMERS PENDING NODES

| ENTITY IDENTIFICATION           | PROTOCOL ROLE | ACCOUNT STATE | DEPLOYMENT DATE | MODERATION ACTIONS |
|---------------------------------|---------------|---------------|-----------------|--------------------|
| chan customer<br>chan@gmail.com | CUSTOMER      | AUTHENTICATED | MAY 24, 2026    |                    |
| chan farmer<br>chan@gmail.com   | FARMER        | AUTHENTICATED | MAY 24, 2026    |                    |
| Joe Farmer<br>farmer@egg.com    | FARMER        | AUTHENTICATED | MAY 24, 2026    |                    |

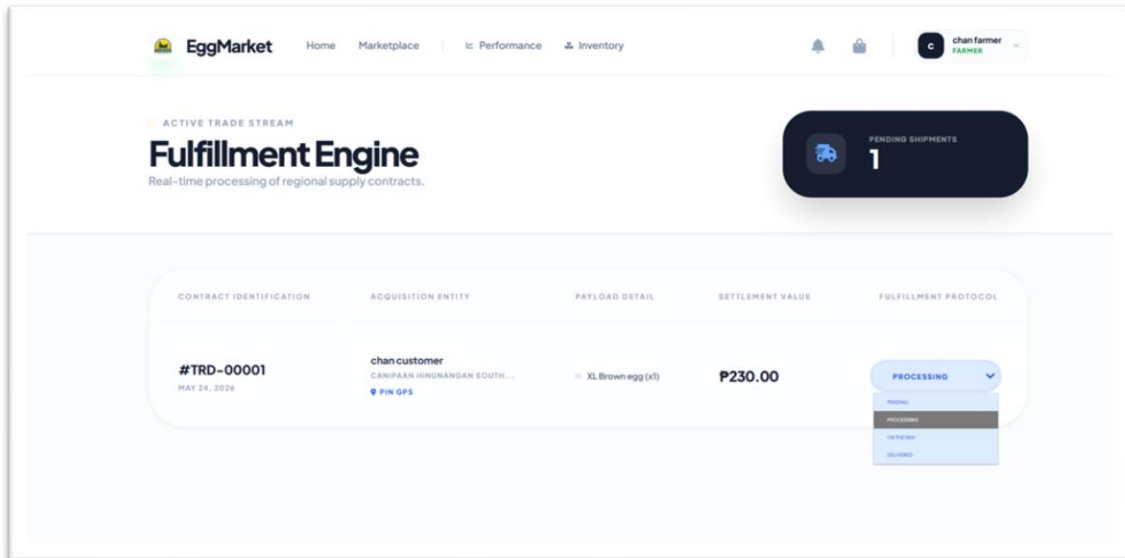
*Figure 12: Admin User Management*



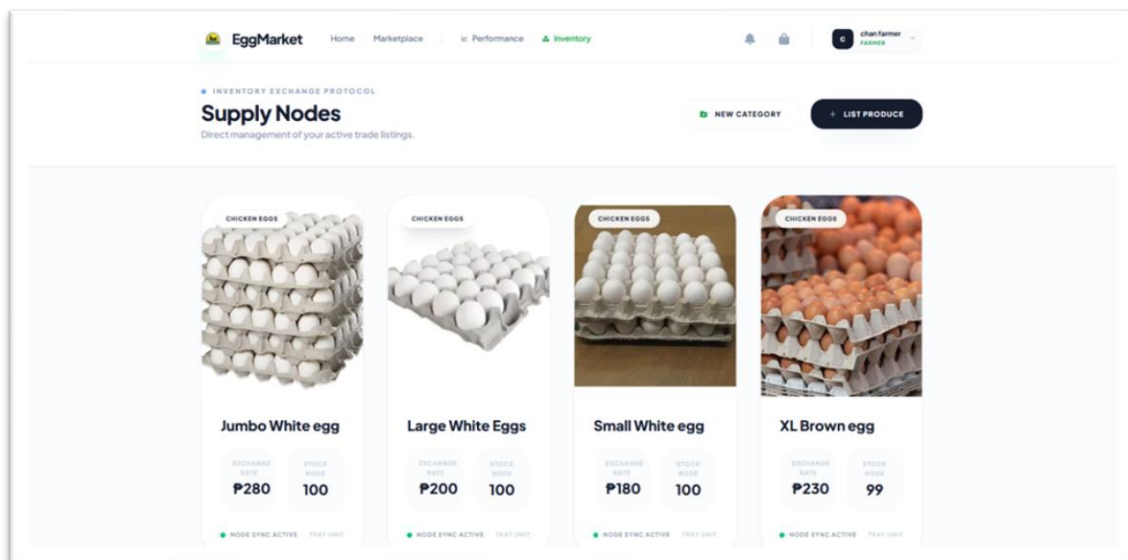
*Figure 13: Admin Settings*



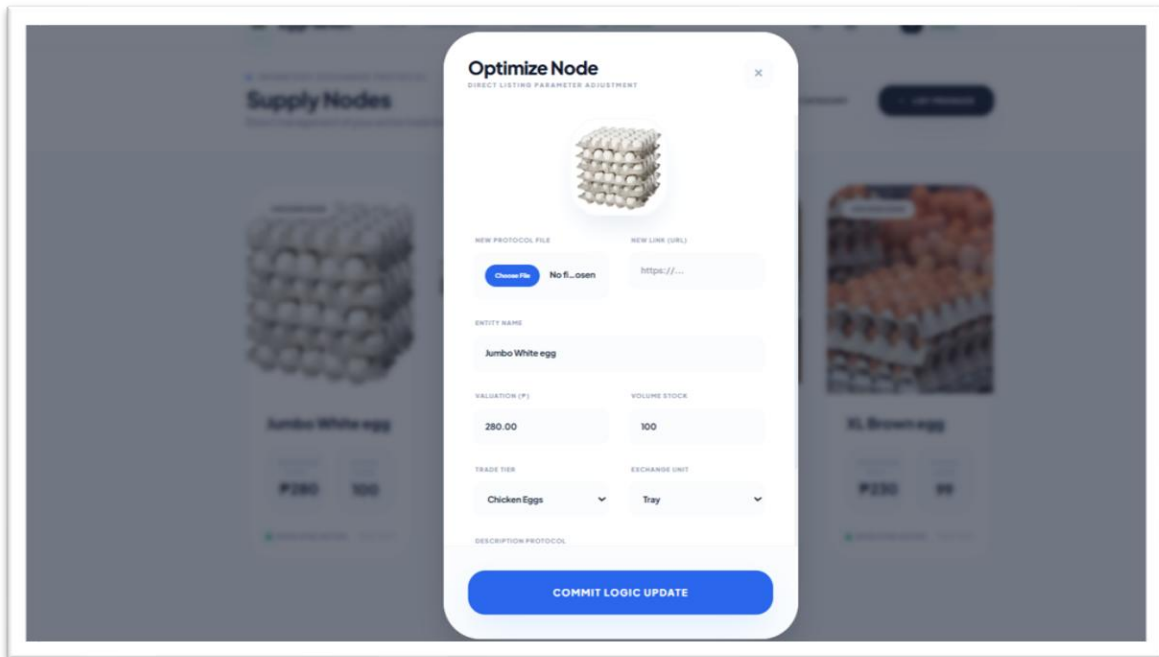
*Figure 14: Farmer Dashboard*



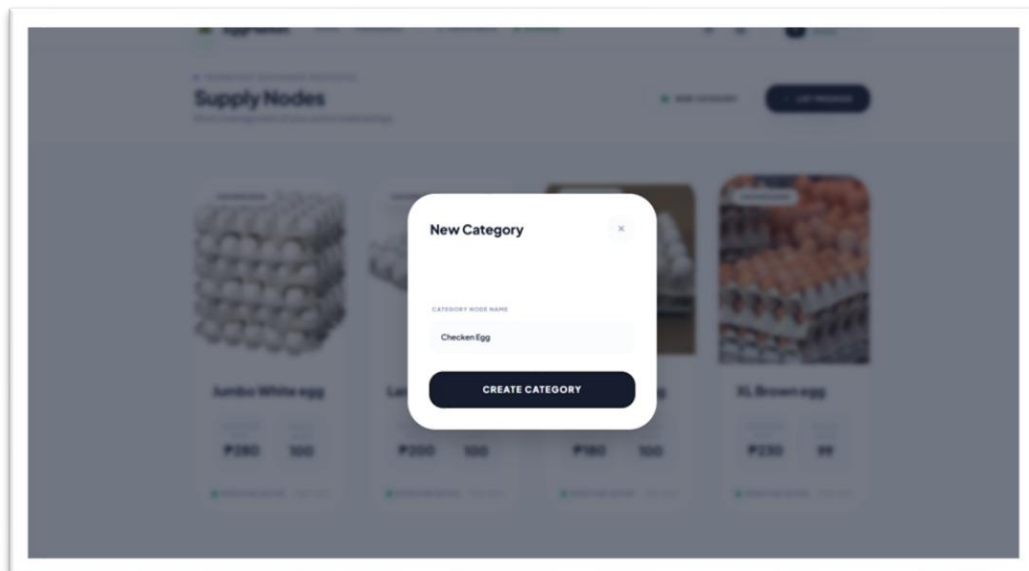
*Figure 15: Farmer Order Management*



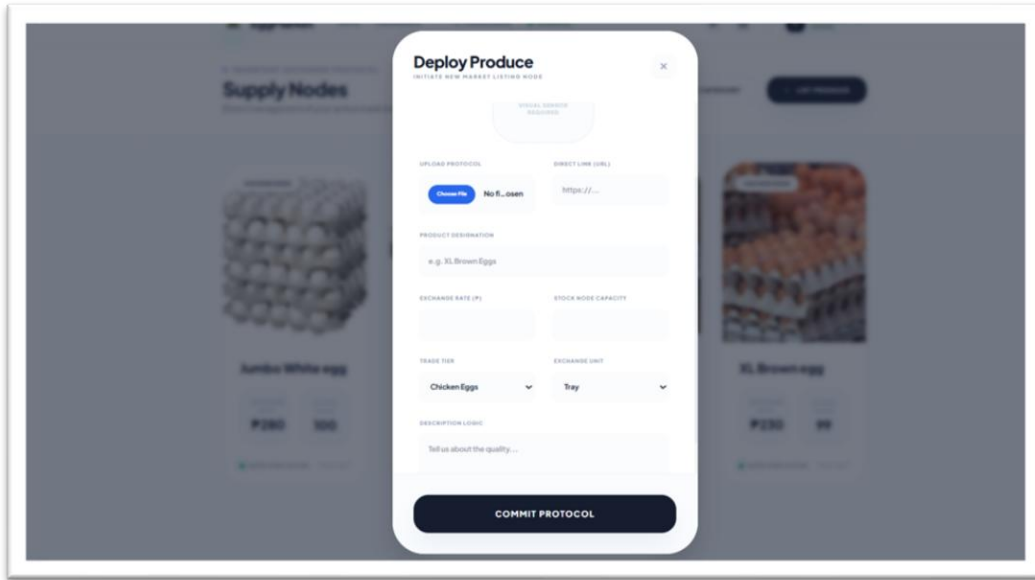
*Figure 16: Farmer Inventory*



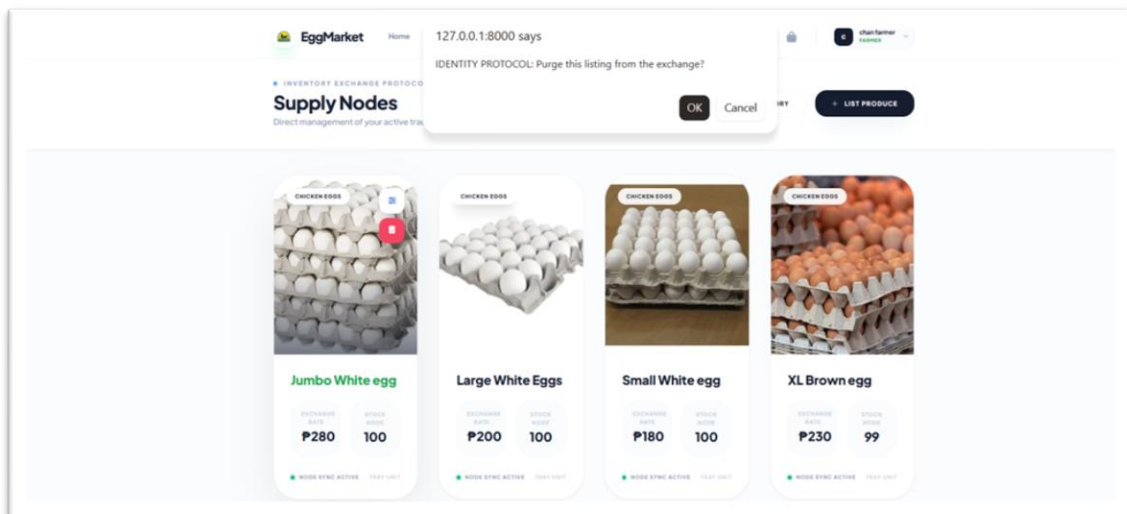
**Figure 17: Farmer Edit Product**



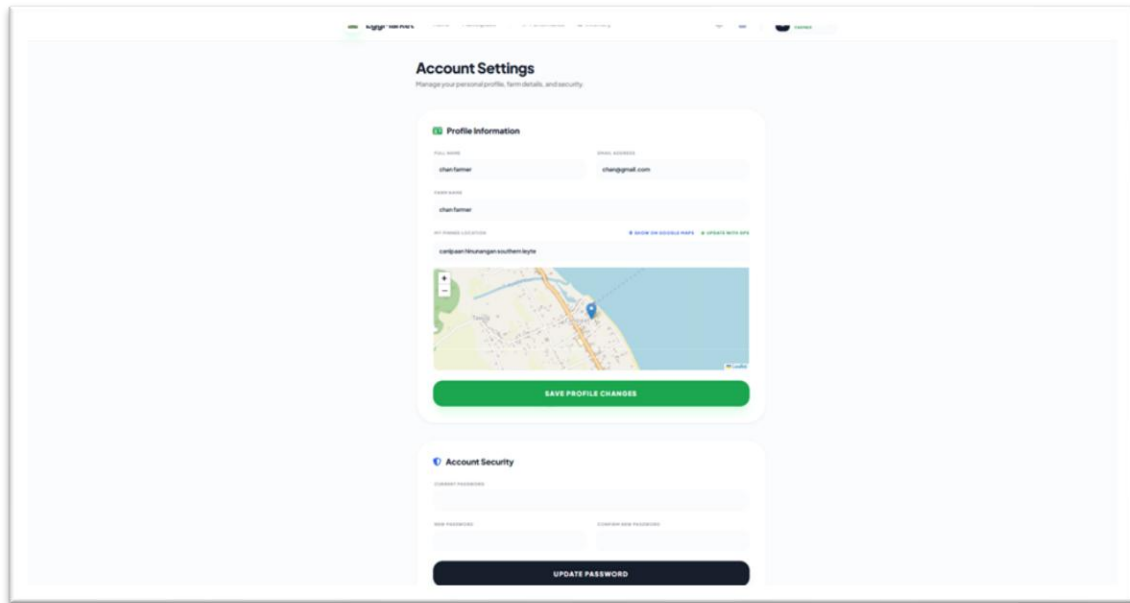
**Figure 18: Farmer add category**



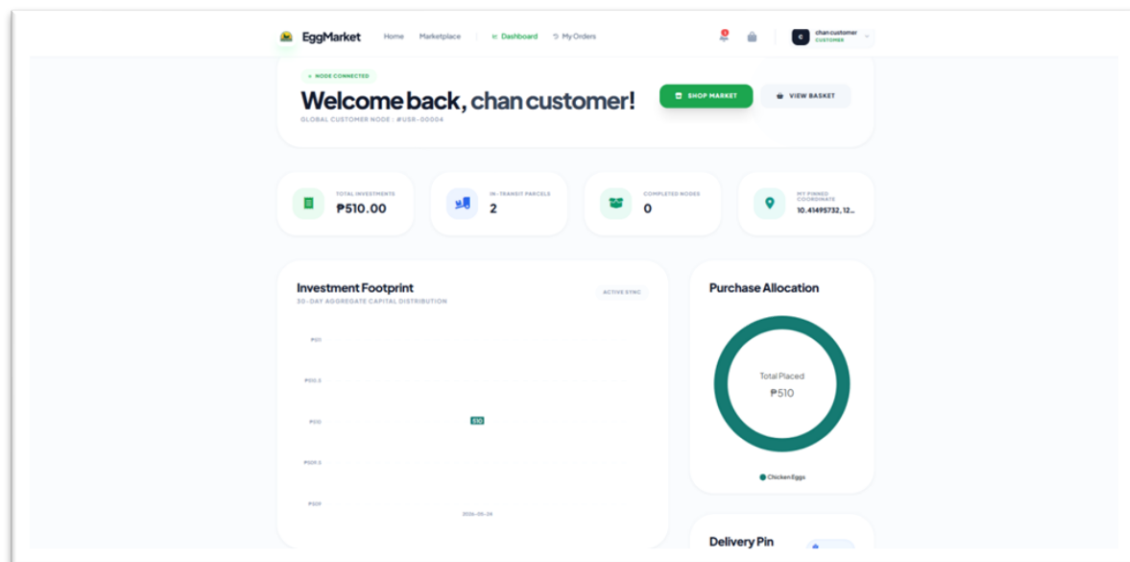
**Figure 19: Farmer add Product**



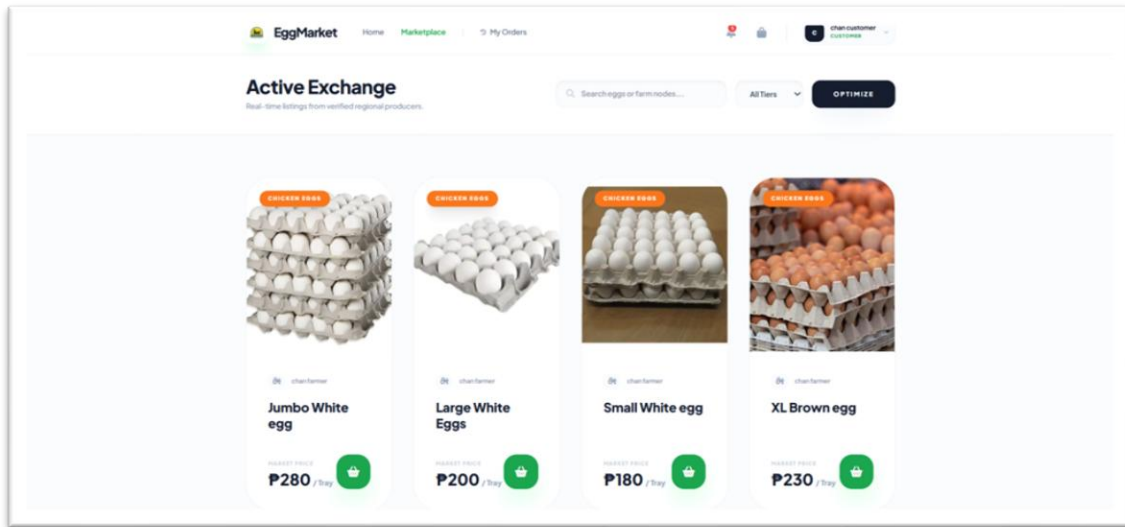
**Figure 20: Farmer delete Product**



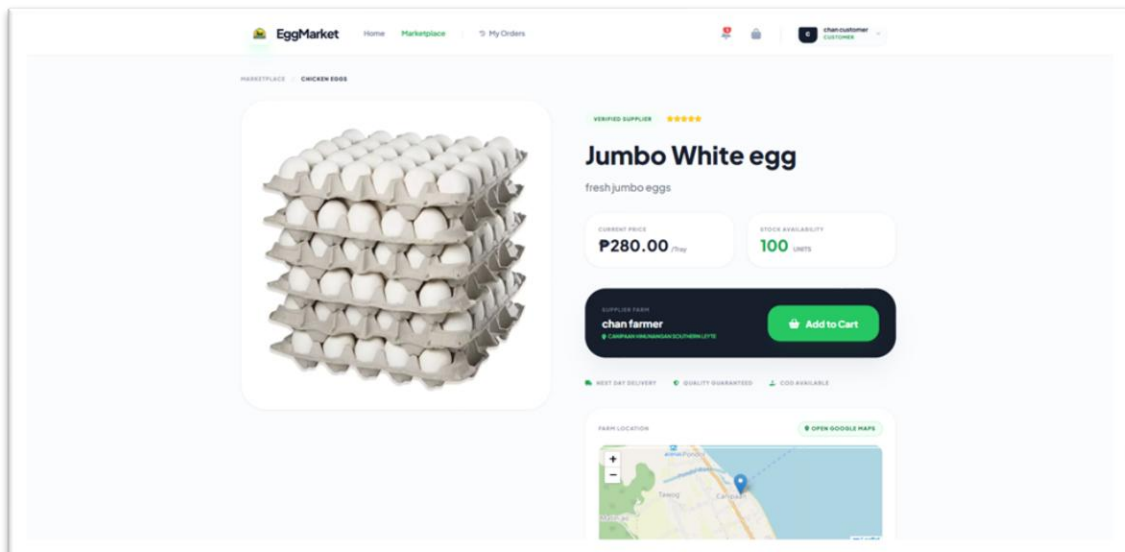
*Figure 21: Farmer Settings*



*Figure 22: Customer Dashboard*



*Figure 23: Customer Marketplace*



*Figure 24: Customer Show Product*

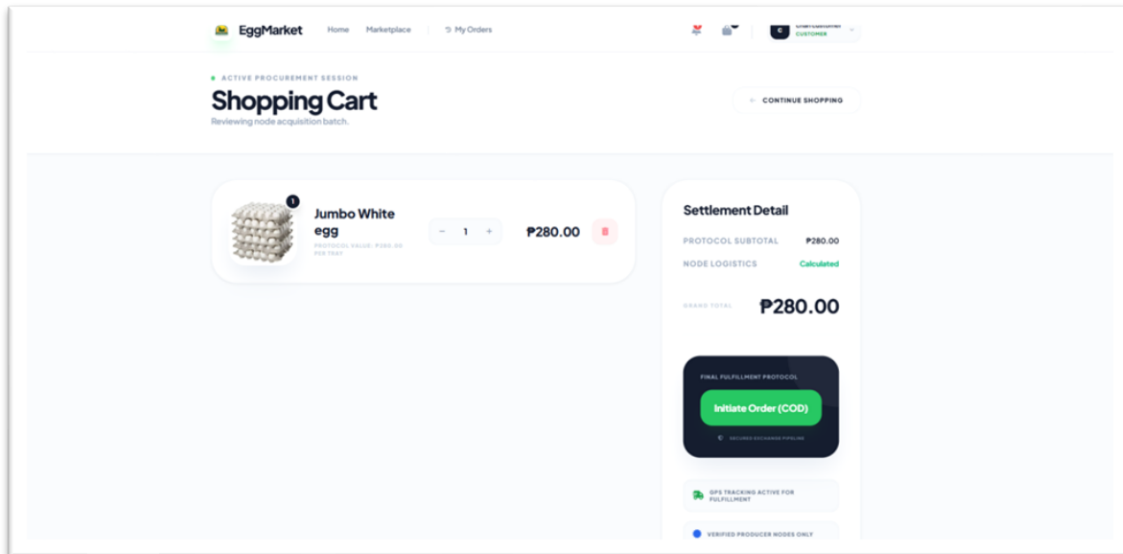


Figure 25: Customer Cart

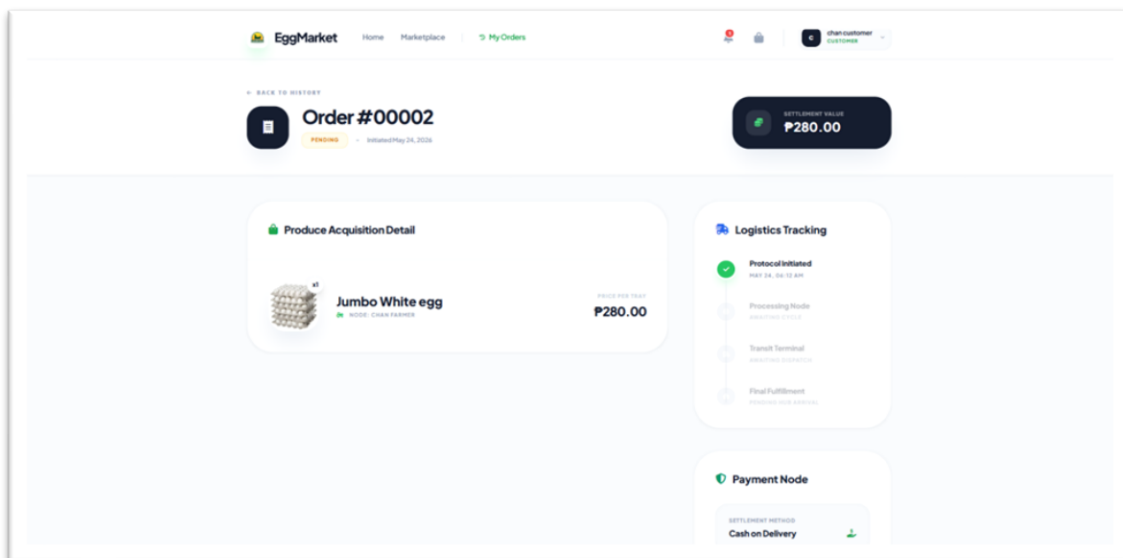
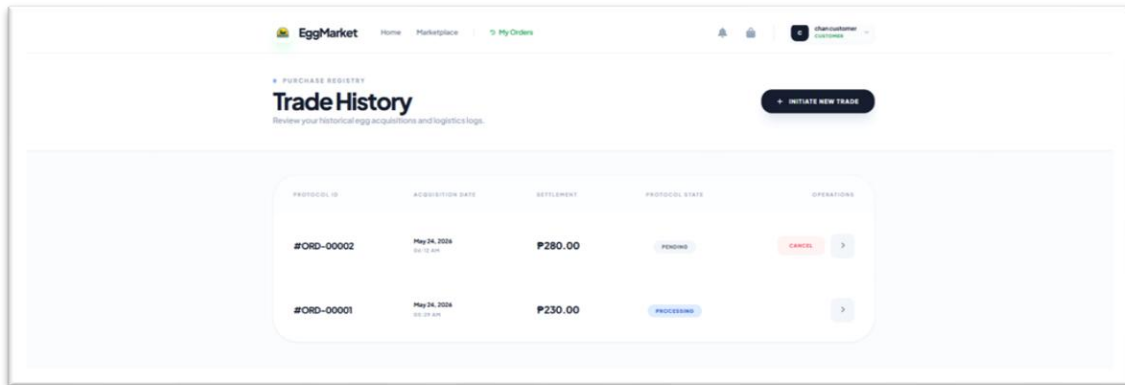
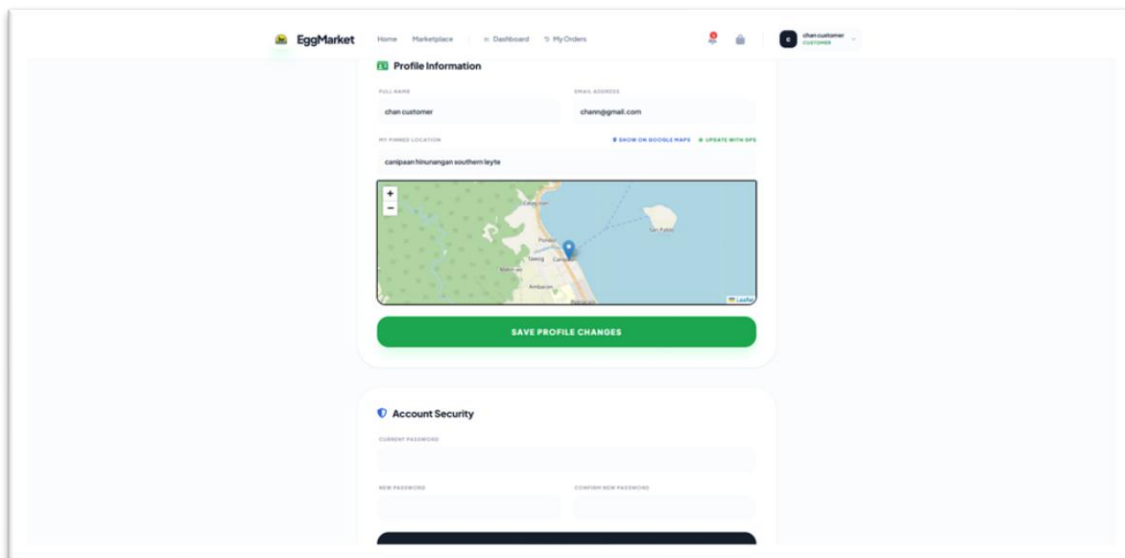


Figure 26: Customer Order-details



**Figure 27: Customer Orders**



**Figure 28: Customer Settings**

## VII. Database Structure

The physical database structure deployed within the Laravel 12 workspace utilizes Eloquent migrations configured for SQLite during testing, and MySQL during production execution. Key tables are described below:

Table: **roles** — user authentication access control

stores user permission grouping types to gate panel access such as admin, farmer, and customer.

| column name | data type       | key type    | nullable | default        | description                                             |
|-------------|-----------------|-------------|----------|----------------|---------------------------------------------------------|
| id          | BIGINT UNSIGNED | PRIMARY KEY | No       | Auto-Increment | Unique role record identifier.                          |
| name        | VARCHAR(255)    | -           | No       | -              | Group title string, such as admin, farmer, or customer. |

Table: **users** — normalized core user store

holds authentication credentials, contact identities, account approval lifecycles, and spatial GIS references.

| column name | data type       | key type    | nullable | default        | description                |
|-------------|-----------------|-------------|----------|----------------|----------------------------|
| id          | BIGINT UNSIGNED | PRIMARY KEY | No       | Auto-Increment | Unique account identifier. |

| column name    | data type       | key type    | nullable | default   | description                                    |
|----------------|-----------------|-------------|----------|-----------|------------------------------------------------|
| name           | VARCHAR(255)    | -           | No       | -         | Authorized legal or corporate name.            |
| email          | VARCHAR(255)    | UNIQUE      | No       | -         | Unique login key and SMTP target.              |
| password       | VARCHAR(255)    | -           | No       | -         | Unidirectional Argon2id/Bcrypt hash.           |
| role_id        | BIGINT UNSIGNED | FOREIGN KEY | No       | -         | Maps to the roles(id) constraint.              |
| status         | VARCHAR(255)    | -           | No       | 'pending' | Vetting state: active, pending, or rejected.   |
| farm_name      | VARCHAR(255)    | -           | Yes      | NULL      | Metadata string for farmer-only profile asset. |
| latitude       | DECIMAL(10, 8)  | -           | Yes      | NULL      | WGS 84 ellipsoid latitude geolocation.         |
| longitude      | DECIMAL(11, 8)  | -           | Yes      | NULL      | WGS 84 ellipsoid longitude geolocation.        |
| address        | VARCHAR(255)    | -           | Yes      | NULL      | Custom descriptive physical drop-off node.     |
| remember_token | VARCHAR(100)    | -           | Yes      | NULL      | Secure static session token.                   |
| created_at     | TIMESTAMP       | -           | Yes      | NULL      | Record generation timestamp.                   |
| updated_at     | TIMESTAMP       | -           | Yes      | NULL      | Dynamic record mutation tracking.              |

Table: **categories** — product classification framework

stores system-wide product classifications such as organic, large eggs, salted eggs, and other product types.

| column name | data type       | key type    | nullable | default        | description                                  |
|-------------|-----------------|-------------|----------|----------------|----------------------------------------------|
| id          | BIGINT UNSIGNED | PRIMARY KEY | No       | Auto-Increment | Unique category identifier index.            |
| name        | VARCHAR(255)    | -           | No       | -              | Readable category descriptive header string. |

Table: **products** — consolidated marketplace catalog

maps items provided by farmers, including active price quotes, stock tracking quantities, and base64 vector imagery.

| column name | data type       | key type    | nullable | default        | description                               |
|-------------|-----------------|-------------|----------|----------------|-------------------------------------------|
| id          | BIGINT UNSIGNED | PRIMARY KEY | No       | Auto-Increment | Unique inventory item identifier.         |
| user_id     | BIGINT UNSIGNED | FOREIGN KEY | No       | -              | Farmer mapping pointing to users(id).     |
| category_id | BIGINT UNSIGNED | FOREIGN KEY | No       | -              | Points to the category in categories(id). |
| name        | VARCHAR(255)    | -           | No       | -              | Human-readable product label.             |
| description | TEXT            | -           | No       | -              | Detailed product specification content.   |
| price       | DECIMAL(10, 2)  | -           | No       | -              | Unit price.                               |
| stock       | INTEGER         | -           | No       | -              | Active ledger volume metrics.             |

| column name | data type    | key type | nullable | default | description                                              |
|-------------|--------------|----------|----------|---------|----------------------------------------------------------|
| unit        | VARCHAR(255) | -        | No       | 'Tray'  | Trading measurement metric, such as tray, box, or piece. |
| image       | LONGTEXT     | -        | Yes      | NULL    | Base64 vector string raw encoding.                       |
| created_at  | TIMESTAMP    | -        | Yes      | NULL    | Creation timestamp.                                      |
| updated_at  | TIMESTAMP    | -        | Yes      | NULL    | Record adjustment tracking.                              |

Table: **orders** — transactional purchase log

stores standard trade execution agreements, aggregated transaction costs, and fulfillment phases.

| column name  | data type       | key type    | nullable | default        | description                                                             |
|--------------|-----------------|-------------|----------|----------------|-------------------------------------------------------------------------|
| id           | BIGINT UNSIGNED | PRIMARY KEY | No       | Auto-Increment | Unique order identification index.                                      |
| user_id      | BIGINT UNSIGNED | FOREIGN KEY | No       | -              | Purchasing customer user mapped to users(id).                           |
| total_amount | DECIMAL(10, 2)  | -           | No       | -              | Net cost total sum of items.                                            |
| status       | VARCHAR(255)    | -           | No       | 'pending'      | Lifecycle stage: pending, processing, shipped, delivered, or cancelled. |
| created_at   | TIMESTAMP       | -           | Yes      | NULL           | Fulfillment registration timestamp.                                     |
| updated_at   | TIMESTAMP       | -           | Yes      | NULL           | Status mutation logging timestamp.                                      |

Table: **order\_items** — order component lines breakdown

associates records with products ordered in unique combinations and enforces price capture snapshots during the order point-in-time to preserve historical values if prices change later.

| column name | data type       | key type    | nullable | default        | description                                                 |
|-------------|-----------------|-------------|----------|----------------|-------------------------------------------------------------|
| id          | BIGINT UNSIGNED | PRIMARY KEY | No       | Auto-Increment | Row key.                                                    |
| order_id    | BIGINT UNSIGNED | FOREIGN KEY | No       | -              | Parent master record in orders(id), with cascade on delete. |
| product_id  | BIGINT UNSIGNED | FOREIGN KEY | No       | -              | Reference to index target in products(id).                  |
| quantity    | INTEGER         | -           | No       | -              | Item multiplier purchased.                                  |
| price       | DECIMAL(10, 2)  | -           | No       | -              | Unit price locked during settlement.                        |

Table: **payments** — financial settlement ledger

secures active micro-payment status vectors matching transactions over multiple payment channels.

| column name | data type       | key type    | nullable | default        | description                                         |
|-------------|-----------------|-------------|----------|----------------|-----------------------------------------------------|
| id          | BIGINT UNSIGNED | PRIMARY KEY | No       | Auto-Increment | Record ledger index.                                |
| order_id    | BIGINT UNSIGNED | FOREIGN KEY | No       | -              | Target order in orders(id), with cascade on delete. |
| method      | VARCHAR(255)    | -           | No       | -              | Payment: COD.                                       |
| status      | VARCHAR(255)    | -           | No       | 'pending'      | Settlement indicator context status.                |
| created_at  | TIMESTAMP       | -           | Yes      | NULL           | Creation timestamp.                                 |
| updated_at  | TIMESTAMP       | -           | Yes      | NULL           | Status change timestamp.                            |

Table: **reviews** — feedback quality audits

allows verified customers to submit feedback on purchased inventory assets to ensure product transparency.

| column name | data type       | key type    | nullable | default        | description                                                    |
|-------------|-----------------|-------------|----------|----------------|----------------------------------------------------------------|
| id          | BIGINT UNSIGNED | PRIMARY KEY | No       | Auto-Increment | Record identifier.                                             |
| user_id     | BIGINT UNSIGNED | FOREIGN KEY | No       | -              | Posting user identifier in users(id), with cascade on delete.  |
| product_id  | BIGINT UNSIGNED | FOREIGN KEY | No       | -              | Targeted product item in products(id), with cascade on delete. |
| rating      | INTEGER         | -           | No       | -              | Metric scale rating indicator range, from 1 to 5 stars.        |
| comment     | TEXT            | -           | No       | -              | Qualitative text statement.                                    |
| created_at  | TIMESTAMP       | -           | Yes      | NULL           | Review timestamp.                                              |
| updated_at  | TIMESTAMP       | -           | Yes      | NULL           | Revision logging and update tracking pointer.                  |

## VIII. Implementation and Testing

### Development Process

The project was developed using an Agile approach, starting with the database schema design, followed by the development of the Laravel backend, and concluding with the high-fidelity UI redesign and real-time API integration

### Testing Procedures

- **Unit Testing:** Verified individual controller methods for product listing and order placement.

- **Integration Testing:** Ensured that stock auto-deducts upon checkout and auto-replenishes upon cancellation.
- **User Acceptance Testing:** Simulated different roles (Admin/Farmer/Customer) to verify RBAC security.

### **Problems Encountered & Solutions**

- **Problem:** Data integrity issues when deleting orders.
- **Solution:** Implemented Database Cascade Deletes and manual child-record removal within PHP transactions.

## **IX. Conclusion**

The Egg Market Management System successfully creates a professional, functional, and efficient digital environment for agricultural trade. By leveraging the robust security and structural efficiency of Laravel 12 alongside modern frontend technologies, the platform effectively bridges the gap between local egg farmers and consumers. It eliminates unnecessary middlemen, allowing producers to list their inventories directly to ensure fresher products at fair prices while maximizing profit margins for the farms.

Beyond simple trade, the system automates the traditionally complex and tedious aspects of supply chain management. It features live inventory tracking that dynamically monitors stock levels, categorizes batches by egg size, and updates availability in real time to prevent manual counting errors or over-ordering. Furthermore, the platform streamlines the entire transaction process by automating order confirmations, invoice generation, and status updates. This centralized digital approach reduces administrative paperwork and provides farmers with clear, data-driven sales insights on a secure, highly scalable infrastructure capable of handling growing trade volumes smoothly.

## **X. Recommendations**

- **Payment Integration:** Future versions should include GCash and PayMaya API integrations for cashless transactions.
- **Mobile App:** Developing a Flutter or React Native mobile application for better on-farm accessibility.

- **AI Forecasting:** Implementing basic machine learning to predict stock demand based on historical sales data.
- **Logistics Tracking:** Real-time driver tracking for orders that are "On the Way."